

Broken Access Control Bug Bounty Guide

IDOR, Privilege Escalation, Force Browsing & Bypass Techniques

IDOR

PrivEsc

Force Browsing

Method Switching

May 2026

Table of Contents

- [1. What is Broken Access Control](#)
- [2. IDOR – Insecure Direct Object References](#)
- [3. Privilege Escalation](#)
- [4. Force Browsing & Path Bypass](#)
- [5. Method-Based Bypasses](#)
- [6. 60 Real-World BAC Issues](#)
- [7. Testing Methodology](#)
- [8. Tools & Automation](#)
- [9. Report Templates](#)
- [10. Quick Cheat Sheet](#)

1. What is Broken Access Control

Broken Access Control (BAC) is the #1 vulnerability on the OWASP Top 10. It occurs when users can act outside their intended permissions – accessing data they should not see, performing actions they are not authorized for, or escalating their privileges.

OWASP Top 10 Ranking

Rank	Vulnerability	% of Apps Affected
#1	Broken Access Control	3.81% of all tested apps
#2	Cryptographic Failures	2.72%
#3	Injection	2.43%
#4	Insecure Design	2.22%
#5	Security Misconfiguration	1.77%

Types of Broken Access Control

Type	Description	Example
IDOR – Read	Access data of other users	Change user_id to view other profiles
IDOR – Modify	Modify data of other users	Change order_id to cancel another order
IDOR – Delete	Delete data belonging to others	Delete other users documents
Horizontal PrivEsc	Access same-role user data	User A accesses User B orders
Vertical PrivEsc	Gain higher role access	User becomes Admin
Force Browsing	Access pages without auth	/admin accessible without login
Method Tampering	Bypass via HTTP method	POST to admin endpoint as GET
Path Traversal	Access paths via traversal	/admin via /admin/ or /ADMIN
Parameter Pollution	Bypass via duplicate params	?role=user&role=admin
Mass Assignment	Set restricted fields	Add is_admin:true on signup
JWT Weakness	Token manipulation	Change alg to none, modify claims
CORS Misconfig	Cross-origin data access	Steal data from any origin

Severity Reference

Finding	Severity	Bounty Range
IDOR (sensitive data: PII, health, financial)	P1-P2	\$1,500-\$15,000
IDOR (read only: settings, activity)	P2-P3	\$500-\$3,000
Privilege escalation (user to admin)	P1-P2	\$2,000-\$15,000

Force browsing (admin panel access)	P1-P2	\$2,000-\$10,000
Force browsing (internal tools)	P2	\$1,000-\$5,000
Method tampering	P2-P3	\$500-\$3,000
Mass assignment	P2	\$1,000-\$5,000
CORS misconfiguration	P2-P3	\$500-\$2,000

Why BAC is #1

- Present in 94% of applications with some form of access control flaw
- Impossible to detect with automated scanners alone
- WAFs provide no protection against BAC
- Affects every application with multiple user roles
- High impact: data breach, admin takeover, financial loss
- Custom to each application - no generic patch pattern

2. IDOR – Insecure Direct Object References

IDOR Attack Patterns

Pattern	Payload	Result
Sequential ID	user_id=1, user_id=2, user_id=3	Access each user data
UUID enumeration	UUID in API response leaks in HTML/JS	Find valid UUIDs
Custom encoding	base64 decode ID, modify, re-encode	Predictable encoded IDs
Wildcard	user_id=* or user_id=all	Return all records
Negative ID	user_id=-1 or user_id=0	May return all records
Array of IDs	user_ids=[1,2,3,4,5]	Batch data extraction
Null/Empty	user_id= or user_id=null	May return default/first user
Self-reference trick	Use own ID in admin context	Access admin data about self
ID type confusion	String ID where int expected: id=admin	Parser confusion
GraphQL alias	query { a:user(id:1){email} b:user(id:2){email} }	Batch IDOR

IDOR Testing Techniques

Technique	How	When to Use
ID increment	Burp Repeater: change id=1 to id=2	Sequential numeric IDs
Batch IDOR	Burp Intruder: payload 1-10000	Numeric IDs, fast enumeration
UUID discovery	Search HTML/JS for UUIDs	UUID-based endpoints
Encoded ID	Decode ID, modify, re-encode	Base64 or custom encoding
Parameter pollution	?id=1&id=2	Duplicate param parsing
Method switching	GET /user/1 -> PUT/DELETE	CRUD endpoints
Replace with own ID	Use attacker ID in victim context	Cross-account access
Wildcards	id=* or id=%	Wildcard-supporting backends

IDOR Parameter Names

Category	Parameters
User	id, user_id, uid, account_id, customer_id, member_id, profile_id
Organization	org_id, team_id, group_id, company_id, workspace_id
Resource	doc_id, file_id, order_id, invoice_id, payment_id
Communication	message_id, thread_id, chat_id, ticket_id, case_id

Financial	transaction_id, txn_id, billing_id, subscription_id
Project	project_id, repo_id, board_id, list_id, card_id
Generic	resource_id, object_id, item_id, entity_id, record_id

3. Privilege Escalation

Vertical Privilege Escalation Techniques

Technique	Payload	Result
Role parameter	role=admin, type=administrator, is_admin=true	Admin role assigned
Mass assignment	{"email":"a@b.com", "role":"admin"}	Admin account created
Hidden field	admin=true, superuser=1, staff=1	Admin features enabled
JWT claim	Modify role claim from user to admin	Token with admin role
Cookie flag	admin_access=true, is_staff=1	Staff access granted
Feature flag	beta=true, internal=true, debug=1	Internal features unlocked
API endpoint	/api/internal/admin, /api/v1/admin	Admin endpoints discovered
GraphQL introspection	__schema mutation discovery	Admin mutations found
Plan upgrade	plan=enterprise, tier=premium	Premium without payment
Permission bits	permissions=0xFFFFFFFF, access=all	All permissions granted

Horizontal Privilege Escalation

Technique	Example	Impact
Organization switching	org_id=1 changed to org_id=2	Access other org data
Workspace jumping	workspace_id changed	Access other workspaces
Team enumeration	team_id incremented	Access other team data
Cross-tenant	tenant_id changed	Multi-tenant data access
Resource ownership	owner_id changed to attacker	Take ownership of resources

4. Force Browsing & Path Bypass

Force Browsing Techniques

Technique	Payload	Result
Direct admin access	GET /admin, /dashboard, /panel	Admin panel exposed
Case variation	/ADMIN, /Admin, /aDmIn	Case-sensitive bypass
Path traversal	/admin/, /admin//, /.admin	Path normalization bypass
Extension tricks	/admin.json, /admin.xml, /admin.html	Content negotiation bypass
API version	/v1/admin, /beta/admin, /internal/admin	Legacy endpoint access
Mobile endpoints	/mobile/admin, /m/admin, /app/admin	Mobile API bypass
Underscore prefix	/_admin, /_internal, /_debug	Hidden endpoints
Dot prefix	/.admin, /.internal, /.env	Hidden path access
URL encoding	/%61dmin (a encoded)	Encoded path bypass
Unicode	/adm%C3%ADn (accented chars)	Unicode normalization bypass

Common Admin/Internal Paths

Category	Paths
Admin	/admin, /administrator, /manage, /management, /dashboard, /panel, /cp, /control
API	/api/internal, /api/private, /api/v1/admin, /api/debug, /api/swagger
Debug	/debug, /test, /testing, /dev, /development, /console, /rails/info
Internal	/internal, /private, /staff, /employee, /backoffice, /backstage
Config	/config, /settings, /env, /.env, /configuration, /setup
Database	/db, /database, /phpmyadmin, /pgadmin, /redis, /mongo-express
Monitoring	/status, /health, /metrics, /prometheus, /grafana, /kibana, /newrelic
Source	/git, /.git, /.svn, /.hg, /backup, /old, /backup.zip, /source.tar.gz
Upload	/upload, /uploads, /files, /media, /assets, /storage
CMS	/wp-admin, /administrator, /admin.php, /admin/login, /user/login

5. Method-Based Bypasses

HTTP Method Switching

Original	Switch To	When It Works
GET (read blocked)	POST, PUT, DELETE, PATCH	Different method bypasses auth
POST (write blocked)	GET, PUT, DELETE, PATCH	GET treated as safe, bypasses auth
DELETE (blocked)	POST with _method=DELETE	Framework method override
PUT (blocked)	POST with X-HTTP-Method-Override: PUT	Method override header
Blocked method	OPTIONS, TRACE, HEAD	Some methods skip auth
POST	POST with Content-Type change	Content-type bypass

Method Override Headers

Header	Value	Target Framework
X-HTTP-Method-Override	PUT, DELETE, PATCH	Generic (Rails, Express, Laravel)
X-HTTP-Method	PUT, DELETE	Generic
X-Method-Override	PUT, DELETE	Generic
_method	PUT, DELETE (as body param)	Laravel, Rails
X-Original-Method	PUT, DELETE	Some proxies
Override	PUT, DELETE	Some APIs

6. 60 Real-World Broken Access Control Issues

IDOR Issues (#1-25)

P2 #1 IDOR – Access Any User Profile

Target: Twitter, LinkedIn style

Steps:

- Login as User A
- GET /api/user/1 returns your profile
- Change to /api/user/2
- Access User B profile with PII

Impact: Read any user profile including email, phone, address

Ref: hackerone.com/reports/1270782 (Twitter), medium.com/@vijayknown/idor-profile

P2 #2 IDOR – Access Any Order/Invoice

Target: E-commerce – Shopify, Amazon style

Steps:

- Place order, note order ID: /order/1001
- Change to /order/1002
- View other customer orders, addresses, payment methods

Impact: Read orders and invoices of all customers

Ref: hackerone.com/reports/769105 (Shopify), medium.com/@securibee/idor-order

P2 #3 IDOR – Download Any Invoice PDF

Target: E-commerce

Steps:

- Download your invoice: /download/invoice/1001
- Change to /download/invoice/1002
- Download any customers invoice PDF

Impact: Unauthorized invoice/financial document access

Ref: hackerone.com/reports/534450, medium.com/@jackhcet/idor-invoice-pdf

P2 #4 IDOR – Read Private Messages

Target: Messaging – Twitter DMs, Slack

Steps:

- Access conversation: /api/messages/12345
- Increment thread ID
- Read private messages between other users

Impact: Private message content exposed

Ref: hackerone.com/reports/1176994 (Twitter), medium.com/@evanricafort/idor-messages

P2 #5 IDOR – Access Private Documents

Target: Google Docs, Dropbox style

Steps:

- Access shared doc: /doc/abc123
- Try sequential IDs: abc122, abc124
- Access unshared documents

Impact: Private documents readable without permission

Ref: hackerone.com/reports/876160 (Google), medium.com/@knownsec/idor-documents

P2 #6 IDOR – Export Other Users Data

Target: Analytics platforms

Steps:

- Export your data: /api/export?user_id=1
- Change user_id to 2
- Export complete data of other users

Impact: Mass data extraction via export IDOR

Ref: hackerone.com/reports/242831 (Twitter), medium.com/@hacker_idor-export

P2 #7 IDOR – Access Other User API Keys

Target: Developer platforms

Steps:

- List your keys: /api/keys/1
- Change to /api/keys/2, /api/keys/3
- Read other developers API keys

Impact: API keys stolen for unauthorized access

Ref: hackerone.com/reports/379288, medium.com/@ajdumanhug/idor-api-keys

P2 #8 IDOR – View Other User Payment Methods

Target: Payment platforms – Stripe

Steps:

- View saved cards: /api/payment_methods/1
- Change ID to access others cards
- View masked card details

Impact: Payment card information exposed

Ref: hackerone.com/reports/732401, medium.com/@cyberfon/idor-payments

P2 #9 IDOR – Access Support Tickets

Target: Zendesk, Intercom style

Steps:

- View your ticket: /ticket/1001
- Change to /ticket/1002

- Read support tickets with attachments

Impact: Private support conversations exposed

Ref: hackerone.com/reports/638120 (Twitter), medium.com/@sajib_h1/idor-tickets

P1 #10 IDOR – View Other Employee Payslips

Target: HR platforms – Workday

Steps:

- View payslip: /payslip/EMP-1001
- Try EMP-1002, EMP-1003
- Access salary data of all employees

Impact: Sensitive compensation data exposed

Ref: hackerone.com/reports/286091, medium.com/@sakurity/idor-payslip

P1 #11 IDOR – Access Health Records

Target: Healthcare platforms

Steps:

- View record: /record/PAT-1001
- Try sequential IDs
- Access other patients medical records

Impact: PHI exposed – HIPAA violation

Ref: hackerone.com/reports/423155, medium.com/@codingo/idor-health

P1 #12 IDOR – View Credit Reports

Target: Financial platforms

Steps:

- View credit report: /credit/USER-1001
- Change user ID
- Access credit reports of other users

Impact: Sensitive financial data exposed

Ref: hackerone.com/reports/510059, medium.com/@sansec/idor-credit

P3 #13 IDOR – Read Other Users Notifications

Target: Social platforms

Steps:

- GET /api/notifications?user_id=1
- Change user_id to 2
- Read activity feed of other users

Impact: Activity data exposed

Ref: hackerone.com/reports/487954, medium.com/@nahamsec/idor-notifications

P2 #14 IDOR – Access Team/Org Member Lists

Target: Slack, Teams style

Steps:

- List team: `/api/team/123/members`
- Change team ID to another team
- List members of private teams

Impact: Private org membership exposed

Ref: hackerone.com/reports/570775 (Slack), medium.com/@stefanofindingidor-team

P2 #15 IDOR - View Other Users Shipping/Address

Target: E-commerce - Uber Eats

Steps:

- View addresses: `/api/addresses/12345`
- Increment ID
- Full address with name and phone

Impact: Physical addresses and phone numbers exposed

Ref: hackerone.com/reports/201983 (Uber), medium.com/@th3g3nt3lmanidor-address

P2 #16 IDOR - Access Meeting Recordings

Target: Zoom, Google Meet

Steps:

- Access recording: `/recording/REC-12345`
- Try sequential REC IDs
- Access private meeting recordings

Impact: Private recordings exposed

Ref: hackerone.com/reports/983201, medium.com/@prateek_0490idor-recordings

P3 #17 IDOR - View Other Users Subscriptions

Target: SaaS platforms

Steps:

- View subscription: `/api/subscription/12345`
- Change ID
- Access billing info of others

Impact: Billing information exposed

Ref: hackerone.com/reports/500901, medium.com/@anasbouzdounidor-subscription

P3 #18 IDOR - Access Delivery Tracking Info

Target: DoorDash, UberEats

Steps:

- Track: `/delivery/ORDER-1001`
- Try ORDER-1002
- Get delivery address, driver info

Impact: Real-time location data exposed

Ref: hackerone.com/reports/387527 (DoorDash), medium.com/@th3g3nt3lman/idor-delivery

P2 #19 IDOR – View KYC/Verification Documents

Target: Identity verification

Steps:

- View doc: `/document/DOC-1001`
- Try sequential IDs
- Access passport, ID cards of others

Impact: Government ID documents exposed

Ref: hackerone.com/reports/447827, medium.com/@l1astbr0th3r/idor-kyc

P2 #20 IDOR – Read Chat/Transcripts

Target: Live chat platforms

Steps:

- View transcript: `/chat/TRANSCRIPT-1001`
- Try sequential IDs
- Read other user support chats

Impact: Private conversations exposed

Ref: hackerone.com/reports/319233, medium.com/@edoverflow/idor-chat

P2 #21 IDOR – Access Private Calendar Events

Target: Google Calendar style

Steps:

- View event: `/event/EVENT-1001`
- Increment ID
- Access private events and attendee lists

Impact: Calendar and attendee data exposed

Ref: hackerone.com/reports/912823, medium.com/@merttasci/idor-calendar

P2 #22 IDOR – View Job Applications of Others

Target: LinkedIn, Indeed

Steps:

- View application: `/application/5001`
- Change to `/application/5002`
- Read resumes of other candidates

Impact: Candidate PII and resumes exposed

Ref: hackerone.com/reports/312948 (LinkedIn), medium.com/@harshbothra/idor-applications

P2 #23 IDOR – Access Private Repos/Source Code

Target: GitHub, GitLab

Steps:

- Try /api/projects/ID without auth
- Access source code of private repos

Impact: Private source code exposed

Ref: hackerone.com/reports/702861 (GitHub), medium.com/@alyssa_herrera/idor-repos

P2 #24 IDOR – View Audit Logs of Other Orgs

Target: Enterprise platforms

Steps:

- Access /api/audit-logs?org_id=1
- Change org_id to 2
- Read audit logs of other organizations

Impact: Security event data exposed

Ref: hackerone.com/reports/692530, medium.com/@jubabosec/idor-audit

P1 #25 IDOR – Modify Other Users Data (Write IDOR)

Target: Generic – profile/settings

Steps:

- PUT /api/user/1 (your profile)
- Change to PUT /api/user/2
- Modify name, email, settings of User B

Impact: Unauthorized modification of user data

Ref: hackerone.com/reports/231981, medium.com/@edoverflow/write-idor

Privilege Escalation (#26–40)

P1 #26 Create Admin Account via Registration

Target: Generic REST APIs

Steps:

- POST /api/register with email, password
- Add: role=admin, is_admin=true
- Account created with admin privileges

Impact: Unauthorized admin account creation

Ref: hackerone.com/reports/687907, medium.com/@sakurity/mass-assignment-admin

P1 #27 Role Parameter Tampering on Profile Update

Target: Generic profile APIs

Steps:

- Login as regular user
- POST /api/profile/update
- Add: role=admin
- Role updated to admin

Impact: Regular user becomes admin

Ref: hackerone.com/reports/376888 (Slack), medium.com/@busesz/role-escalation

P1 #28 JWT Algorithm None – Auth Bypass

Target: JWT-based auth

Steps:

- Decode JWT, change alg to none
- Remove signature
- Send modified token
- Authentication bypassed

Impact: Forge tokens for any user

Ref: hackerone.com/reports/226856 (Auth0), medium.com/@adam.juel/jwt-none

P1 #29 JWT RS256 to HS256 Algorithm Switch

Target: JWT with asymmetric keys

Steps:

- Change alg from RS256 to HS256
- Sign with public key as HMAC secret
- Forge admin token

Impact: Token forgery using public key

Ref: hackerone.com/reports/226856, medium.com/@brompwnie/jwt-alg-switch

P2 #30 Feature Flag Enable for Premium

Target: SaaS – Medium, Spotify

Steps:

- POST /api/preferences with: premium=true, beta=true
- Premium features unlocked

Impact: Premium features without payment

Ref: hackerone.com/reports/425362, medium.com/@circlvr/feature-flag-premium

P2 #31 Staff Mode via Cookie Manipulation

Target: Twitter, Reddit style

Steps:

- Add cookie: staff_mode=1, is_staff=true
- Access staff-only features

Impact: Staff features via cookie tampering

Ref: hackerone.com/reports/1008026 (Twitter), medium.com/@rajstarcx/staff-mode

P1 #32 Plan Upgrade Without Payment

Target: SaaS – Slack, Notion

Steps:

- POST /api/subscription/upgrade with plan=enterprise

- Server upgrades without payment verification

Impact: Premium without payment

Ref: hackerone.com/reports/441163, medium.com/@anshuman_bh/plan-upgrade

P1 #33 Add Self to Admin Group

Target: Google Workspace, Okta style

Steps:

- POST /api/groups/ADMIN-GROUP/members
- Add your user ID
- Join admin group

Impact: Join admin groups without auth

Ref: hackerone.com/reports/384918 (Slack), medium.com/@busesz/admin-group

P1 #34 GraphQL Admin Mutation Without Auth

Target: GraphQL APIs

Steps:

- Introspect schema for admin mutations
- Execute deleteUser as regular user
- Admin action succeeds

Impact: Admin mutations accessible

Ref: hackerone.com/reports/745903, medium.com/@graphql-hacker/admin-mutation

P2 #35 Old API Version Lacks Authorization

Target: Twitter API v1 style

Steps:

- /v2/admin/users returns 403
- /v1/admin/users returns 200
- Legacy API lacks security

Impact: Admin data via legacy API

Ref: hackerone.com/reports/769105 (Shopify), medium.com/@anshuman_bh/old-api

P2 #36 Bypass Approval Workflow

Target: Enterprise expense management

Steps:

- POST directly to /request/approve with approved=true
- Skip manager approval

Impact: Sensitive actions without approval

Ref: hackerone.com/reports/231549, medium.com/@bugbounty/approval-bypass

P1 #37 Change Account Owner

Target: SaaS team accounts

Steps:

- POST /api/team/settings with owner_id=YOUR_ID
- Become team owner

Impact: Organization takeover

Ref: hackerone.com/reports/403050 (Slack), [medium.com/@busesz owner-change](https://medium.com/@busesz/owner-change)

P1 **#38 Admin Impersonation via X-Impersonate Header**

Target: Enterprise APIs

Steps:

- Add header: X-Impersonate-User: admin@target.com
- Requests processed as admin

Impact: Full admin impersonation

Ref: hackerone.com/reports/278279, [medium.com/@vijayknown impersonate](https://medium.com/@vijayknown/impersonate)

P1 **#39 KYC Bypass for Restricted Features**

Target: Financial/Crypto

Steps:

- POST /api/kyc/verify with status=verified
- Access trading without KYC

Impact: Financial features without identity verification

Ref: hackerone.com/reports/417614, [medium.com/@vulnerability kyc-bypass](https://medium.com/@vulnerability/kyc-bypass)

P1 **#40 Promote to Super Admin via Hidden Parameter**

Target: Multi-level admin

Steps:

- POST /api/user/promote with level=super_admin
- Gain super admin access

Impact: Super admin access

Ref: hackerone.com/reports/239481, [medium.com/@detectify super-admin](https://medium.com/@detectify/super-admin)

Force Browsing & Path Bypass (#41-50)

P1 **#41 Admin Panel Accessible Without Authentication**

Target: Generic web apps

Steps:

- Visit /admin while logged out
- No redirect to login
- Full admin panel accessible

Impact: Unauthenticated admin access

Ref: hackerone.com/reports/419883, [medium.com/@hussainsenyolt unauth-admin](https://medium.com/@hussainsenyolt/unauth-admin)

P2 #42 Internal API Docs Exposed

Target: Swagger/OpenAPI

Steps:

- Visit `/swagger-ui.html`, `/api-docs`, `/v2/api-docs`
- Full API documentation exposed

Impact: API schema and endpoints leaked

Ref: hackerone.com/reports/264330, medium.com/@seanmelia/swagger-exposed

P2 #43 Debug Endpoints Accessible in Production

Target: Generic

Steps:

- Visit `/debug`, `/test`, `/console`, `/rails/info`
- Debug features exposed

Impact: Debug information and routes exposed

Ref: hackerone.com/reports/286048, medium.com/@detectify/debug-exposed

P2 #44 Source Code Backup Files Accessible

Target: Generic

Steps:

- Try `/backup.zip`, `/source.tar.gz`, `/app.zip`
- Download complete source code

Impact: Source code exposed

Ref: hackerone.com/reports/286048, medium.com/@detectify/source-backup

P2 #45 Database Admin Panels Exposed

Target: phpMyAdmin, pgAdmin

Steps:

- Visit `/phpmyadmin`, `/pgadmin`, `/adminer`
- Database management accessible

Impact: Direct database access

Ref: hackerone.com/reports/264330, medium.com/@seanmelia/db-admin

P2 #46 Monitoring Dashboards Unprotected

Target: Prometheus, Grafana, Kibana

Steps:

- Visit `/prometheus`, `/grafana`, `/kibana`, `/metrics`
- Internal metrics exposed

Impact: System internals and metrics exposed

Ref: hackerone.com/reports/264330, medium.com/@seanmelia/monitoring

P2 #47 CMS Admin Login Unprotected

Target: WordPress, Joomla

Steps:

- Visit `/wp-admin/`, `/administrator`
- No IP restriction or 2FA on admin login

Impact: Admin login brute-forceable

Ref: hackerone.com/reports/419883, medium.com/@hussainsenyolt/cms-admin

P2 #48 Version Control Exposed

Target: Git, SVN

Steps:

- Visit `/.git/`, `/.svn/`
- Repository contents accessible
- Use `git-dumper` to extract

Impact: Full source code and history exposed

Ref: hackerone.com/reports/286048, medium.com/@detectify/git-exposed

P2 #49 Environment Variables Exposed

Target: Generic

Steps:

- Visit `/.env/`, `/config.php/`, `/settings.py`
- Database passwords, API keys exposed

Impact: Secrets and credentials exposed

Ref: hackerone.com/reports/286048, medium.com/@detectify/env-exposed

P2 #50 Case-Insensitive Path Bypass

Target: Generic

Steps:

- `/ADMIN` blocked, try `/admin/`, `/Admin/`, `/aDmIn`
- One variant bypasses auth

Impact: Auth bypass via case variation

Ref: hackerone.com/reports/264330, medium.com/@seanmelia/case-bypass

Method & Parameter Bypass (#51-60)

P2 #51 Auth Bypass via X-Original-URL

Target: IIS/ASP.NET

Steps:

- `/admin` blocked
- Add: `X-Original-URL: /admin` to request to `/`
- IIS rewrites to `admin` without auth check

Impact: Access restricted endpoints via IIS rewrite

Ref: hackerone.com/reports/219011, [medium.com/@_security_researcher x-original-url](https://medium.com/@_security_researcher/x-original-url)

P2 #52 Auth Bypass via X-Rewrite-URL

Target: Reverse proxy

Steps:

- Add: X-Rewrite-URL: /admin
- Proxy rewrites to admin without auth

Impact: Bypass auth via rewrite header

Ref: hackerone.com/reports/250853, [medium.com/@regala x-rewrite-url](https://medium.com/@regala/x-rewrite-url)

P2 #53 Method Override POST to DELETE

Target: Generic REST

Steps:

- DELETE /api/post/1 blocked
- POST /api/post/1 with X-HTTP-Method-Override: DELETE
- Post deleted

Impact: Bypass method restrictions

Ref: hackerone.com/reports/209283, [medium.com/@thevillagehacker method-override](https://medium.com/@thevillagehacker/method-override)

P2 #54 Parameter Pollution Role Bypass

Target: Generic REST

Steps:

- POST /api/profile?role=user&role=admin
- Backend parses second parameter
- Admin role applied

Impact: Role bypass via duplicate params

Ref: hackerone.com/reports/226293, [medium.com/@edoverflow param-pollution](https://medium.com/@edoverflow/param-pollution)

P1 #55 Mass Assignment on Registration

Target: Generic REST APIs

Steps:

- POST /register with {email, password, role: admin, is_verified: true}
- Server saves all fields

Impact: Admin account with verified status

Ref: hackerone.com/reports/687907, [medium.com/@sakurity mass-assignment](https://medium.com/@sakurity/mass-assignment)

P2 #56 IP Allowlist Bypass via X-Forwarded-For

Target: Admin panels

Steps:

- /admin blocked from external IP

- Add: X-Forwarded-For: 127.0.0.1
- Access granted

Impact: Bypass IP-based restrictions

Ref: hackerone.com/reports/268086, medium.com/@knownsec/xff-bypass

P2 #57 CORS Misconfig for Admin API Access

Target: APIs

Steps:

- Access-Control-Allow-Origin: *
- Any website can call admin API
- Steal admin data cross-origin

Impact: Cross-origin admin data theft

Ref: hackerone.com/reports/317931 (Twitter), medium.com/@stefanofinding/cors-admin

P2 #58 OPTIONS Method Bypasses Auth Check

Target: Generic

Steps:

- GET /admin returns 403
- OPTIONS /admin returns 200 with allowed methods
- Endpoint enumeration without auth

Impact: Enumerate endpoints without auth

Ref: hackerone.com/reports/286048, medium.com/@detectify/options-bypass

P2 #59 GraphQL Introspection Exposes Admin Schema

Target: GraphQL APIs

Steps:

- Query __schema for types/mutations
- Full schema including admin operations exposed

Impact: Complete API schema leaked

Ref: hackerone.com/reports/745903, medium.com/@graphql-hacker/introspection

P2 #60 API Version Bypass - v1 Lacks Auth

Target: Twitter API style

Steps:

- /v2/admin/delete requires auth
- /v1/admin/delete no auth
- Admin action via legacy

Impact: Legacy endpoint lacks security

Ref: hackerone.com/reports/769105 (Shopify), medium.com/@anshuman_bh/api-version

7. Testing Methodology

Phase 1: Recon & Mapping

Step	Action	Tool
1	Map all endpoints via crawling	Burp Spider, ffuf
2	Identify ID patterns in URLs	Burp Logger++
3	Map user roles and permissions	Manual testing
4	Find all API versions	ffuf, Burp
5	Discover hidden params	Arjun, Param Miner
6	Check for GraphQL introspection	GraphQLmap, curl
7	Gather wordlists for admin paths	SecLists

Phase 2: IDOR Testing

Step	Action	Expected
1	Increment numeric IDs: 1,2,3,4,5	Access other users data
2	Batch test: Burp Intruder 1-1000	Mass enumeration
3	Test all CRUD: GET/PUT/DELETE/POST	Write/delete IDOR
4	Test encoded IDs: base64 decode + modify	Encoded ID access
5	Try wildcards: id=* or id=%	Return all records
6	Try negative/null: id=-1, id=0	Default/first record
7	GraphQL alias batch query	Batch extraction

Phase 3: Privilege Escalation Testing

Step	Action	Payload
1	Test role parameter on registration	role=admin
2	Test role on profile update	role=administrator
3	Test JWT manipulation	alg:none, alg:HS256
4	Test feature flags	beta=true, premium=true
5	Test plan/tier parameters	plan=enterprise
6	Test mass assignment	Add restricted fields
7	Test admin endpoint as user	GET /admin/users
8	Test cookie flags	admin=1, staff=true

9	Test GraphQL admin mutations	Introspect + execute
10	Test all API versions	/v1/ vs /v2/

Order: 1) IDOR (highest payout, easiest) 2) Privilege Escalation 3) Force Browsing 4) Method/Header bypasses.
Always start with simple ID increment.

8. Tools & Automation

Essential BAC Testing Tools

Tool	Purpose	Command/Usage
Burp Authorize	Automated auth testing	Test endpoint access as different roles
Burp AutoRepeater	Automated replacement	Replace IDs across all requests
Burp Param Miner	Hidden param discovery	Discover hidden parameters
ffuf	Directory/API discovery	ffuf -u URL/FUZZ -w wordlist.txt
Arjun	HTTP param discovery	arjun -u URL -m GET,POST
AuthMatrix	Authorization testing	Role-based access matrix
jwt_tool	JWT testing	jwt_tool.py token_here
GraphQLmap	GraphQL testing	graphqlmap -u URL/graphql
Nuclei	BAC templates	nuclei -u target -t exposed-panels/
Git-dumper	Git extraction	git-dumper URL/.git/ output/
Waybackurls	Historical URLs	waybackurls target.com grep admin
Gau	URL enumeration	gau target.com grep api.*admin

Bash One-Liners

Purpose	Command
IDOR numeric scan	for i in \$(seq 1 100); do curl -s -H "Cookie: S" "URL/user/\$i" grep -q email && echo "Found:\$i"; done
Admin path discovery	ffuf -u URL/FUZZ -w seclists/admin-urls.txt -mc 200
Parameter discovery	arjun -u URL/api/user -m GET -w role,admin
JWT none test	jwt_tool.py token_here -X a
Method switch test	for m in GET POST PUT DELETE; do curl -X \$m -s -w "\$m:%{http_code}" "URL/admin"; done
Force browse admin	for p in admin dashboard panel manage; do curl -s -w "\$p:%{http_code}" "URL/\$p"; done
CORS test	curl -I -H "Origin:https://evil.com" URL grep -i access-control
IP bypass test	curl -H "X-Forwarded-For:127.0.0.1" -s -w "%{http_code}" "URL/admin"

9. Report Templates

Template: IDOR – Sensitive Data Exposure

Section	Content
Title	Sensitive Data Exposure via IDOR in <code>/api/user/{id}</code>
Severity	High (CVSS 7.5)
Description	The <code>/api/user/{user_id}</code> endpoint does not verify that the authenticated user owns the requested <code>user_id</code> . Changing <code>user_id</code> exposes any user profile.
Steps	1. Login as User A 2. GET <code>/api/user/1</code> (your profile) 3. Change to <code>/api/user/2</code> 4. Receive User B profile with email, phone, address
Impact	Any user profile accessible including PII. Affects all users.
Fix	Verify <code>user_id</code> matches session user before returning data. Use server-side session, not client-supplied ID.

Template: Admin Panel Unauthenticated Access

Section	Content
Title	Admin Panel Accessible Without Authentication
Severity	Critical (CVSS 9.1)
Description	The <code>/admin</code> endpoint is accessible without authentication, allowing anyone to view and manage the application.
Steps	1. Visit <code>target.com/admin</code> while logged out 2. Observe admin dashboard loads without login 3. Can view user list, settings, logs
Impact	Full administrative access without credentials. Complete data breach and system compromise possible.
Fix	Require authentication and admin role verification on all admin endpoints. Implement middleware check.

Bounty Reference

Finding	HackerOne	Bugcrowd
IDOR (sensitive PII)	\$500-\$10,000	\$300-\$5,000
IDOR (modify/delete)	\$1,500-\$15,000	\$1,000-\$10,000
Privilege escalation (admin)	\$2,000-\$15,000	\$1,500-\$10,000

Unauthenticated admin	\$2,000-\$10,000	\$1,500-\$8,000
Force browsing (internal tools)	\$1,000-\$5,000	\$500-\$3,000
Mass assignment	\$1,000-\$5,000	\$500-\$3,000

10. Quick Cheat Sheet

BAC Testing Checklist

#	Test	Payload/Action
1	ID increment	Change id=1 to id=2,3,4,5
2	ID batch test	Intruder: ids 1-1000
3	ID method switch	GET /user/1 -> PUT/DELETE
4	Encoded ID	base64 decode, modify, re-encode
5	Wildcard ID	id=* or id=%
6	Negative/null ID	id=-1, id=0, id=null
7	GraphQL alias IDOR	query { a:user(1){email} b:user(2){email} }
8	Role escalation	role=admin on register/update
9	JWT alg:none	Change alg to none, remove signature
10	JWT alg switch	RS256 to HS256
11	Mass assignment	Add is_admin, role fields
12	Feature flags	beta=true, premium=true
13	Cookie manipulation	admin=1, staff=true
14	Force browse admin	/admin, /dashboard, /panel
15	Force browse internal	/internal, /debug, /api/private
16	API version bypass	/v1/admin vs /v2/admin
17	Method override	X-HTTP-Method-Override: DELETE
18	X-Original-URL bypass	X-Original-URL: /admin
19	X-Forwarded-For bypass	X-Forwarded-For: 127.0.0.1
20	Parameter pollution	?role=user&role=admin
21	CORS misconfig	Origin: evil.com, check response
22	GraphQL introspection	__schema query
23	Case variation	/ADMIN, /Admin, /aDmIn
24	Path traversal	/admin/, /admin//, ./admin
25	Hidden endpoints	/_admin, /.env, /config

Admin Path Wordlist (Top 50)

admin, administrator, manage, management, manager, dashboard, panel, control, cp, backend, backoffice, backstage, internal, private, staff,

```
employee, root, superuser, sysadmin, webadmin, master, operator, service,  
support, helpdesk, moderator, mod, agent, developer, dev, debug, test,  
testing, console, api-admin, api/internal, api/private, api/debug, v1/admin,  
v2/admin, beta/admin, mobile/admin, app/admin, m/admin, admin.php, admin.py,  
admin.aspx, admin.jsp, admin.cgi, admin/login, admin/auth, admin/signin
```

Golden Rule: If you can change an ID and access data that is not yours, it is an IDOR. If you can access /admin without being an admin, it is force browsing. If you can add role=admin and become admin, it is mass assignment. Test these three on every application - they are the most common and highest-paying BAC vulnerabilities.